SurgeTix: Product Requirements & System Architecture

Nov 4, 2026 · @Varshil

Overview

SurgeTix is a flash-sale ticketing backend that stays correct and responsive when demand exceeds supply by 100× or more, all at once. It sells seats for one event to millions of simultaneous buyers, never sells a seat twice, and keeps the buying path fast by letting only as many users in as the system can serve.

The problem. Flash sales break ticketing platforms. In November 2022, 3.5 million fans registered for the Eras Tour presale and about 14 million people (including bots) tried to buy; Ticketmaster cancelled the general sale (source). In September 2024, more than 13 million fans queued for about 1.5 lakh Coldplay Mumbai tickets; BookMyShow crashed at opening and resale listings hit 10–20× face value within an hour (source).

The approach. A layered funnel: cheap layers (CDN, rate limits, a virtual waiting room) absorb the crowd, and only an admitted, bounded stream reaches the correctness-critical booking core, where seat holds are atomic database operations.

What makes it stand out. Queue status served almost entirely from cache (a signed queue ticket plus one global admission counter), a server-side best-available seat picker that cuts hold conflicts, low-contention general admission through bucketed counters, cancellations with automatic refunds whose returned seats go straight to a second-chance queue, short-lived rotating QR codes with replay-resistant one-time check-in, and a nightly audit that checks money and seats still reconcile.

Scope of this doc. Product requirements, capacity targets, architecture, data model, APIs, key flows, failure handling, test plan and build phases for a portfolio-grade implementation in Go. Payments are simulated with a fake provider that behaves like a real one; no real money moves.

Goals, non-goals and success metrics

Success means zero double-bookings under any load, a flat booking-path p99 during a spike, and a measured improvement over a naive baseline under identical load.

Goals

1. Make double-selling a seat or general-admission ticket impossible within the data model (conditional updates, check and unique constraints), and verify it with repeated concurrency tests.
2. Keep the booking path responsive during a spike by admitting users at a rate the system can serve, adjusted to live health.
3. Give every buyer a fair, transparent experience: a queue position, an ETA, a clear sold-out signal, and a second chance when seats come back.
4. Never double-charge, never refund a confirmed booking by mistake, and never leave a paid buyer without a ticket or a refund.
5. Support the full ticket lifecycle: purchase, cancellation with refund, transfer to a friend, and online entry scan.
6. Maximize efficiency: the fewest database operations, network calls and servers per ticket sold, measured and reported.
7. Measure the impact of each layer against a naive baseline under an identical load profile.

Non-goals

- Real payments, real money or PCI compliance (FakePay stands in, including refunds).
- Multi-region deployment, dynamic pricing, or a paid resale marketplace (free transfer to a friend is in scope).
- A polished consumer frontend (a thin React app exists only to demo the flows).
- Machine-learning bot detection (proof-of-work, limits and heuristics only).

Success metrics (targets; real values come from load tests)

Metric	Definition	Target
Double-booked seats or oversold GA tickets	Audit query after every test run	0 in every run
Seat-hold API latency	`POST /holds` request to response, at the admitted rate	p99 < 300 ms
Checkout API latency	`POST /checkout` request to response (order + payment intent created; excludes the buyer paying)	p99 < 300 ms

Metric	Definition	Target
Payment completion latency	Buyer submits payment on FakePay to signed webhook received (provider-controlled; healthy FakePay)	p99 < 2 s
Booking confirmation latency	Webhook received to order `CONFIRMED` with tickets issued	p99 < 500 ms
Refund completion latency	Cancellation accepted to refund `COMPLETED`	Healthy provider: p99 < 60 s. Provider down: tickets voided immediately, refund completes within 15 min of provider recovery
Queue status latency and cache share	`GET /queue/state`	p99 < 50 ms; ⩾ 95% answered by cache
Hold conflict rate	409 responses ÷ hold attempts	< 5% with best-available; baseline measured
Postgres round trips per seat sold	Counted from query metrics, hold through confirmation	⩽ 4
Buyer-facing error rate	(5xx + timeouts) ÷ buyer requests, same k6 profile as the naive baseline (T6)	⩾ 10× lower than baseline
Wrong refunds	Refunds issued for orders that end `CONFIRMED`	0 (nightly audit)
Paid orders without a ticket or refund	Nightly audit after recovery	0

Users and key journeys

Three actors shape the design: the buyer we serve, the organizer who runs a sale, and the bot we must keep out.

Actor	Wants	Design consequence
Buyer	A fair shot at seats, clear status, no double charge, easy cancellation	Waiting room with position and ETA, best-available picker, atomic holds, idempotent checkout, self-serve cancel and refund
Organizer / admin	Create an event, set sale time, limits and refund policy, watch and control the sale live	Admin API, per-event config, pause and resume, live control-room dashboard
Gate staff	Validate tickets quickly at entry	Online scan endpoint (gates need venue connectivity; offline mode is out of scope), rotating QR codes, one-time check-in
Bot / scalper	Grab inventory faster than humans and resell screenshots	Proof-of-work on queue join, one queue spot per verified account, purchase caps, rotating QR codes, transfer limits

Buyer journey (happy path)

1. Signs in (email/password or Google) before the sale.
2. Opens the event page at 11:55; joins the waiting room. Users who join before 12:00 get a random position; later arrivals queue in order.
3. Sees "You're #18,342, about 6 minutes" updating every few seconds.
4. Gets admitted with a short-lived access token; the seat map loads.
5. Picks up to 4 seats; they are held for 10 minutes with a visible countdown.
6. Pays through the fake provider; receives a confirmation and a QR ticket by email.

Unhappy paths that must feel clean: seat taken while choosing (409 with nearby alternatives, or one tap for best-available), hold expires during payment (seats released, any late payment refunded automatically), sold out while waiting (everyone in line is told at once and can join the second-chance queue), and a change of plans (cancel before the cutoff, refund within a minute, ticket invalidated, seat re-offered).

Functional requirements

Everything below is in scope. P0 proves the core claim; P1 completes the ticket lifecycle and the efficiency features that make the project stand out.

ID	Requirement	Priority
FR-1	Sign up and sign in with email/password (bcrypt) and Google OAuth with PKCE; RS256 JWTs verified via JWKS; email verification required before joining a queue	P0
FR-2	Organizer creates an event with a venue, reserved sections (rows, seats, price tiers) and general-admission sections, sale start time, per-user limit and refund policy	P0
FR-3	Buyer views event details and a seat map with near-real-time availability	P0
FR-4	Buyer joins a per-event waiting room and receives a signed queue ticket; position and ETA are computed from one cached global counter	P0
FR-5	Admission controller admits users at a rate that adapts to booking latency and database load; admitted users exchange their queue ticket for a short-lived access token	P0
FR-6	Admitted buyer holds 1–4 specific seats atomically (all or nothing) for 10 minutes	P0
FR-7	Best-available: buyer asks for N seats in a section or price tier and the server picks adjacent free seats, skipping locked rows	P0
FR-8	General admission: buyer holds N tickets from a GA section without choosing seats, never oversold	P0
FR-9	Holds expire automatically and inventory returns to sale	P0

ID	Requirement	Priority
FR-10	Checkout a hold: create order, pay via FakePay, confirm on signed webhook; refund automatically only when confirmation is definitively rejected, never on a timeout	P0
FR-11	Confirmed booking issues tickets with rotating QR codes (time-based codes derived from a per-ticket secret)	P0
FR-12	Every state-changing endpoint accepts an Idempotency-Key	P0
FR-13	Buyer cancels a booking (whole or some tickets) before the event's cutoff; refund amount follows the event's policy (for example 100% until 7 days out, 50% until 24 hours, then none)	P1
FR-14	Refunds run as a saga: invalidate tickets, refund through FakePay, return inventory, notify; exactly once even under retries	P1
FR-15	Second-chance queue: after sell-out, users can stay in line; inventory returned by expiry, failed payment or cancellation is offered to them in order	P1
FR-16	Ticket transfer to another verified account (max 1 transfer per ticket, disabled within 24 hours of the event) with the old QR secret revoked	P1
FR-17	Gate scan (online; gates need connectivity): staff app validates a QR code and checks the ticket in exactly once	P1
FR-18	Bot defense: proof-of-work challenge on queue join, per-account and per-IP limits, purchase caps per account and payment method	P1
FR-19	Live seat-map deltas pushed to viewers over SSE	P1
FR-20	Admin control room: queue length, admission rate, seats left, errors; pause, resume and kill switch per sale	P1

ID	Requirement	Priority
FR-21	Ticket confirmation and refund emails sent asynchronously	P1
FR-22	Nightly audit: every BOOKED seat has one valid ticket, every paid order has tickets or a refund, ledger totals match FakePay	P1
FR-23	Buyer releases a hold early	P1

Non-functional requirements

Correctness of seat state and money beats availability; everything else (seat maps, queue position, emails) may be eventually consistent.

Area	Requirement
Consistency	Seat state, GA counts, holds, orders, refunds and ticket ownership strongly consistent (single Postgres primary, ACID). Seat map and queue position eventually consistent, ⩽ 2 s stale
Latency	Per-step targets as defined under Success metrics (hold, checkout API, payment completion, confirmation, refund); seat-map read p99 < 100 ms; online gate scan p99 < 150 ms. Provider-dependent steps have an SLO for a healthy provider and a recovery objective for an unavailable one
Throughput	Booking core sized for a configured admitted rate (target 1,000 holds/s on a laptop-class cluster); waiting room handles the full crowd
Efficiency	⩽ 4 Postgres round trips per seat sold; one Redis call per queue join; ⩾ 95% of queue polls answered by cache; seat-map pushes as deltas, not full maps; all services built with PGO and allocation-checked hot paths

Area	Requirement
Availability	No single point of failure in the request path on Kubernetes (≥ 2 replicas per stateless service); graceful degradation if notifications, analytics or SSE fail
Durability	No confirmed booking, refund or captured payment lost; Kafka `acks=all`, outbox for every event
Idempotency	Retries of hold, checkout, cancel, refund, transfer, payment webhook and event consumption produce exactly one effect
Fairness	Pre-sale arrivals randomized; post-start FIFO; one active queue entry per account; second-chance offers strictly in queue order
Security	JWT on all buyer endpoints; signed queue tickets and admission tokens bound to user and event; proof-of-work on join; per-user and per-IP rate limits; webhook signatures; rotating QR secrets never sent in plain form; parameterized SQL
Observability	RED metrics per service, traces across gateway → services → DB → Kafka, structured logs with trace IDs, efficiency metrics (DB round trips per sale, cache hit ratio)
Operability	One-command local run (Docker Compose); Kubernetes manifests with probes, HPA, PodDisruptionBudgets and graceful shutdown; per-sale kill switch

Capacity estimates

The crowd is huge but the inventory is tiny, so the hard load is queue polling, not booking: the booking core only ever sees the admitted rate.

Reference sale (design target): one 50,000-seat stadium, 1,000,000 buyers arriving within 5 minutes, max 4 seats per buyer, about 2.5 seats per order (≈ 20,000 orders). 1M is the target this design is built and tested for. 10M is a stretch goal: each event's queue

counters live on one Redis shard, so adding application replicas does not spread that load (see below).

Quantity	Estimate	Drives
Queue joins	1M ÷ 300 s ≈ 3,300/s (33,000/s at 10M: stretch, single Redis shard, measure before claiming)	One Redis `INCR` / `ZADD` per join, after a proof-of-work check
Queue status polls	Adaptive interval (5 s near the front, 30 s far back) ≈ 40,000/s from clients	Answered by one global counter cached at the edge for 1 s: origin sees about 1 request per edge location per second
Admitted users	Adaptive, start at 500/s	Bounds everything below it
Hold attempts	500/s × ~1.1 attempts with best-available ≈ 550/s; target capacity 1,000/s	Postgres primary sizing, connection pool
GA ticket holds	Up to 1,000/s on one section	Spread over 32 counter buckets so no single row is hot
Seat-map reads	Every admitted user, several refreshes ≈ 2,000/s	Redis snapshot per section; SSE deltas for live viewers
Checkout + payment	≈ 20,000 orders over the sale, peak ~200/s	Saga throughput, FakePay limits
Cancellations and refunds	Assume 3% of orders ≈ 600, bursty after event changes	Refund saga, second-chance offers
Gate scans	50,000 entries over 2 hours, peak ~30/s	Scan endpoint, check-in uniqueness
Kafka events	Holds, expiries, confirmations, refunds ≈ 2,000/s peak	1 broker locally, 3 on Kubernetes
Data size	Seats 50K rows ≈ 10 MB; queue 1M entries ≈ 100 MB in Redis	Everything fits in memory

The per-event Redis hot spot. The queue sequence, the admission counter and the per-event keys share one hash slot (`{event}` hash tag) so Lua scripts can update them atomically. That pins one event's queue traffic to a single Redis shard no matter how many

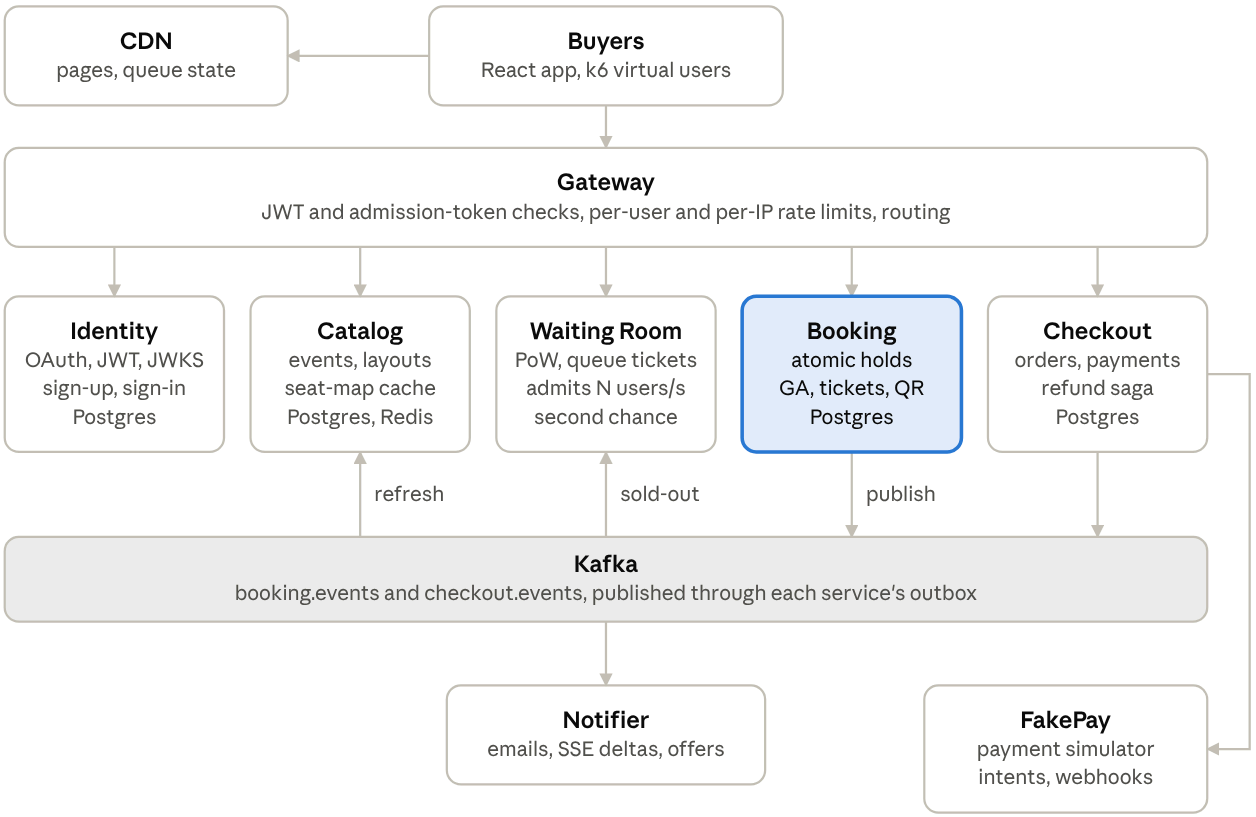
Waiting Room replicas run. Version 1 accepts this and measures it: benchmark one event's queue at increasing join and poll rates and record Redis CPU, commands per second, memory, p99 latency and failover time. Only if the measured ceiling is too low do we design partitioned queues with a globally coordinated admission policy, a much harder problem deliberately left out of v1.

Load-test target: what a single machine cannot generate, a k6 run spread across 2–3 cloud VMs can: aim for 50,000 virtual users polling the queue plus a separate contention test of 5,000 concurrent buyers on the same 500 seats.

Architecture overview

The system is a funnel: the CDN, gateway and waiting room absorb the crowd, and only admitted buyers reach Booking, the one service that must be strictly correct.

Only admitted buyers reach the booking core; slow work leaves through Kafka



SurgeTix architecture · 6 services, 1 worker, Kafka backbone

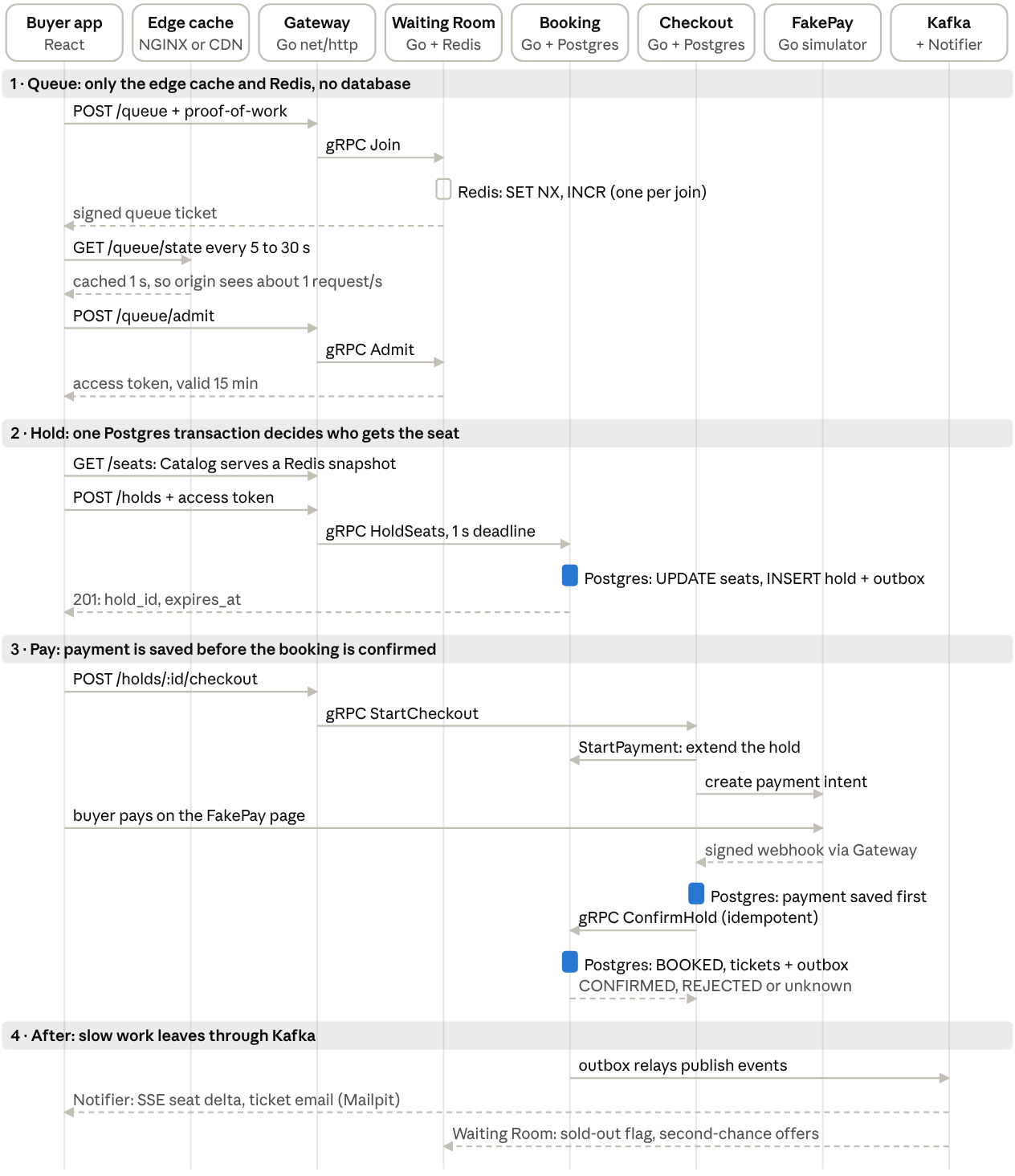
Every browser request enters through the gateway; services call each other over gRPC, and anything slow or fan-out (emails, seat-map refresh, sold-out signals) travels as Kafka events. FakePay's webhooks return through the gateway to Checkout.

Data flow

A buyer reaches Postgres only after admission: the queue runs on the edge cache and Redis, one transaction decides each hold, and everything slow travels through Kafka.

The crowd stays in the cache and Redis; only admitted buyers reach Postgres

Solid: request · dashed: response or event · filled mark: Postgres write, the source of truth



Data flow of one purchase · 4 phases, 27 steps

Steps run top to bottom. Identity (JWT keys, cached by the Gateway) and Catalog (seat maps) sit behind the Gateway and are left out to keep the picture readable; the Tech stack table covers them.

Tech stack

Every tool is free, runs on a laptop, and was picked for one job; each row names the realistic alternative and why it lost.

Layer	Choice	Where it's used	Why	Why not the alternatives
Language	Go 1.27	All services, workers, FakePay	Cheap goroutine per request, small memory and images, fast startup, strong standard library (`net/http`, `crypto`, `context`)	Java/Spring: heavier, slower startup. Node: one thread for CPU work such as JWT signing and proof-of-work checks. Rust: slow to build with while learning
Frontend	React + TypeScript (Vite)	Buyer app, admin control room, gate scanner page	You already know it; it only needs to demo the flows	Next.js: server rendering not needed. Native mobile app: out of scope
Edge cache	NGINX `proxy_cache`, 1 s microcache (a CDN such as Cloudflare if hosted)	`GET /queue/state`, static assets	One config block absorbs nearly every queue poll	Varnish: equivalent, one more tool. Per-pod cache in Go only: origin load grows with pod count
Gateway	Go `net/http` (pattern routing)	Single public entry: JWT and access-token checks, rate limits, REST to gRPC	Full control, and it is the logic interviewers ask about	Kong, Envoy, Traefik: production-grade but hide the interesting code; name them as the production swap
Service calls	gRPC + Protobuf (`buf` for lint and breaking-change checks)	Gateway to services, Checkout to Booking	Typed contracts, deadlines that propagate, HTTP/2 multiplexing	REST/JSON: no contracts, no deadline propagation. GraphQL: built for clients, not service calls. Connect-RPC: a fine alternative, less recognised

Layer	Choice	Where it's used	Why	Why not the alternatives
Source of truth	PostgreSQL 17, one database per service; `pgx` + `sqlc`; `goose` migrations	Seats, GA buckets, holds, bookings, tickets, orders, payments, refunds, sagas, idempotency, outbox	Row locks, conditional updates, CHECK and partial unique constraints, `SKIP LOCKED`	MySQL: no partial indexes. MongoDB: document transactions are heavier for multi-seat updates. Cassandra: no multi-row transactions. CockroachDB/Spanner: distributed SQL adds latency the 50,000-seat inventory does not need. GORM: hides the SQL you must control for locking
In-memory state	Redis 7 (`go-redis`, Lua scripts)	Queue tickets and counters, admission lease and epoch, proof-of-work nonces, seat bitmaps, seat-map snapshots, rate limits	Sub-millisecond atomic operations, TTLs, sorted sets, Lua for check-and-set	Memcached: no scripts or sorted sets. Postgres for every poll: tens of thousands of queries per second on the database. etcd for the lease: correct but a whole cluster for one key. Valkey: a drop-in open-source fork, either works
Event log	Apache Kafka (KRaft mode) + `franz-go`	Outbox relays publish; Notifier, Catalog and Waiting Room consume	Durable, replayable, many consumer groups, per-key ordering	RabbitMQ: strong for job queues, no replay. NATS JetStream: lighter and reasonable, less recognised. Redis Streams: would add load to the Redis already busy with the queue. SQS/SNS: cloud-only
Payments	FakePay (own Go simulator: intents, signed webhooks, refunds, idempotency keys)	Checkout	Latency, failures and lost webhooks on demand, at load	Stripe or Razorpay test mode: realistic but rate-limited and cannot fail on command; can be swapped in behind the same interface later
Auth	Identity service: `bcrypt`, `golang-jwt` (RS256), `x/oauth2` (Google, PKCE), JWKS	Sign-up, sign-in, verification at the gateway	Shows the security depth from AuthPlatform without a separate project	Keycloak: heavy to run locally. Auth0/Firebase: hosted, hides the work

Layer	Choice	Where it's used	Why	Why not the alternatives
Live updates	Server-Sent Events	Seat-map deltas, second-chance offers	One-way push over plain HTTP with built-in reconnect	WebSockets: two-way not needed, more connection state. Polling: wasteful at this scale
Email	Mailpit (local SMTP inbox)	Ticket and refund emails	Free, inspectable during demos	SendGrid/SES: real delivery not needed; swappable
Observability	Prometheus + Grafana; OpenTelemetry SDK, Collector and Tempo; `slog` JSON logs (Loki optional)	Every service; the Surge Control Room dashboard	Open-source standard, runs locally, one trace across services	ELK: heavy on memory. Datadog/New Relic: paid. Jaeger: fine instead of Tempo
Load testing	k6 (arrival-rate executors), spread across VMs for big runs	Tests T1 to T13	JavaScript scripts, efficient Go engine, fixed request rates	JMeter: heavy and GUI-oriented. Locust: lower throughput per machine. Gatling: JVM tooling
Fault injection	Toxiproxy, `kubectl delete pod`, `SIGSTOP`	T4, T5, T12, T13	Precise faults such as "commit, then drop the response"	Chaos Mesh: powerful, heavy for v1
Testing	`go test -race`, `testcontainers-go`, table-driven and fuzz tests	Every package	Real Postgres, Redis and Kafka catch locking bugs mocks cannot	Mocks only: miss SQL and lock behaviour
Run and deploy	Docker Compose locally; Kubernetes (kind or k3d) with CloudNativePG, Strimzi and Redis Sentinel; GitHub Actions CI	Local development, high-availability demo, CI	Standard, free, transfers to any company	Nomad/ECS: less common. Managed cloud databases: cost

Services and responsibilities

Six Go services plus one worker, each split for a stated reason: different scaling needs, a separate failure domain, or separate data ownership. The new lifecycle features land inside existing services rather than adding more boxes.

Service	Owns	Why it is separate	Scales on
Gateway (HTTP)	Routing, JWT and admission-token verification, rate limits, request IDs, cacheable queue-status route	Single public entry; stateless edge logic	Request rate
Identity	Users, credentials, email verification, OAuth (Google, PKCE), JWT signing keys, JWKS	Security boundary; keys never leave it	Sign-ins (low)
Catalog	Events, venues, seat maps, sale and refund policy config; seat-map snapshots in Redis	Read-heavy; cached aggressively	Seat-map reads
Waiting Room	Proof-of-work challenges, signed queue tickets, admission controller, access tokens, second-chance queue	Faces the whole crowd; must scale independently of booking	Queue joins
Booking	Seat and GA inventory, best-available picker, holds, expiry, tickets, rotating QR secrets, transfers, gate check-in	Correctness core for inventory and ticket ownership	Admitted rate
Checkout	Orders, checkout and refund sagas, cancellations, payments, FakePay webhooks, nightly audit	Money path isolated from inventory failures	Orders
Notifier (worker)	Ticket and refund emails, SSE seat-map deltas, second-chance offers	Slow side effects off the critical path	Kafka lag

A FakePay simulator plays the external payment provider (payment intents, delays, failures, signed webhooks).

Technology choices, where each is used, and the alternatives rejected are in Tech stack above.

Data ownership: each service owns its schema (separate Postgres databases on one instance locally); no service reads another's tables. Cross-service needs go through gRPC or Kafka events.

Data model

The `seats` table is the source of truth for inventory; a seat's status changes only through conditional updates, and a unique constraint on tickets is the final guard.

PostgreSQL (one database per service)

```
-- booking_db ------------------------------------------------------
CREATE TABLE seats (
  event_id        UUID   NOT NULL,
  seat_id         TEXT   NOT NULL,              -- e.g. 'A-12'
  section         TEXT   NOT NULL,
  row_label       TEXT   NOT NULL,
  seat_number     INT    NOT NULL,              -- adjacency for best-
available
  price_paise     BIGINT NOT NULL CHECK (price_paise >= 0),
  status          TEXT   NOT NULL DEFAULT 'AVAILABLE'
                  CHECK (status IN ('AVAILABLE','HELD','BOOKED')),
  hold_id         UUID,
  hold_expires_at TIMESTAMPTZ,  -- mirror of holds.expires_at, written ONLY
in the same transaction
  PRIMARY KEY (event_id, seat_id),
  CHECK ((status = 'AVAILABLE') = (hold_id IS NULL)),
  CHECK ((status = 'HELD') = (hold_expires_at IS NOT NULL))
) WITH (fillfactor = 80);                       -- room for HOT updates on
status changes
CREATE INDEX seats_held_expiry ON seats (hold_expires_at) WHERE status =
'HELD';
CREATE INDEX seats_free_by_row ON seats (event_id, section, row_label,
seat_number) WHERE status = 'AVAILABLE';

CREATE TABLE ga_buckets (
  event_id  UUID     NOT NULL,
  section   TEXT     NOT NULL,
  bucket    SMALLINT NOT NULL,                  -- 0..31
```

```
  capacity  INT      NOT NULL CHECK (capacity >= 0),
  remaining INT      NOT NULL CHECK (remaining >= 0 AND remaining <=
capacity),
  PRIMARY KEY (event_id, section, bucket)
);

CREATE TABLE holds (
  hold_id    UUID PRIMARY KEY,
  event_id   UUID NOT NULL,
  user_id    UUID NOT NULL,
  kind       TEXT NOT NULL CHECK (kind IN ('SEATS','GA')),
  status     TEXT NOT NULL CHECK (status IN
('ACTIVE','PAYMENT_PENDING','CONVERTED','EXPIRED','RELEASED')),
  expires_at TIMESTAMPTZ NOT NULL,              -- CANONICAL hold expiry
  order_id   UUID UNIQUE,                       -- set when checkout starts
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX holds_expiring ON holds (expires_at) WHERE status IN
('ACTIVE','PAYMENT_PENDING');

CREATE TABLE hold_seats (                       -- normalized seats of a hold
  hold_id  UUID NOT NULL REFERENCES holds,
  event_id UUID NOT NULL,
  seat_id  TEXT NOT NULL,
  PRIMARY KEY (hold_id, seat_id),
  FOREIGN KEY (event_id, seat_id) REFERENCES seats
);

CREATE TABLE hold_ga_allocations (              -- exact quantity taken from
each bucket
  hold_id     UUID     NOT NULL REFERENCES holds,
  event_id    UUID     NOT NULL,
  section     TEXT     NOT NULL,
  bucket      SMALLINT NOT NULL,
  quantity    INT      NOT NULL CHECK (quantity > 0),
  released_at TIMESTAMPTZ,                      -- set once when restored;
prevents double release
  PRIMARY KEY (hold_id, section, bucket),
  FOREIGN KEY (event_id, section, bucket) REFERENCES ga_buckets
);

CREATE TABLE bookings (                         -- makes ConfirmHold
idempotent
```

```
  booking_id   UUID PRIMARY KEY,
  hold_id      UUID NOT NULL UNIQUE REFERENCES holds,
  order_id     UUID NOT NULL UNIQUE,
  user_id      UUID NOT NULL,
  confirmed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE tickets (
  ticket_id      UUID PRIMARY KEY,
  booking_id     UUID NOT NULL REFERENCES bookings,
  event_id       UUID NOT NULL,
  seat_id        TEXT,                          -- NULL for GA
  section        TEXT NOT NULL,
  owner_user_id  UUID NOT NULL,
  status         TEXT NOT NULL CHECK (status IN
('VALID','CANCELLED','CHECKED_IN')),
  qr_secret_enc  BYTEA NOT NULL,
  transfer_count SMALLINT NOT NULL DEFAULT 0 CHECK (transfer_count BETWEEN 0
AND 1),
  checked_in_at  TIMESTAMPTZ,
  CHECK ((status = 'CHECKED_IN') = (checked_in_at IS NOT NULL)),
  FOREIGN KEY (event_id, seat_id) REFERENCES seats
);
CREATE UNIQUE INDEX one_live_ticket_per_seat ON tickets (event_id, seat_id)
  WHERE seat_id IS NOT NULL AND status <> 'CANCELLED';

CREATE TABLE ticket_transfers (
  transfer_id UUID PRIMARY KEY,
  ticket_id   UUID NOT NULL UNIQUE REFERENCES tickets,  -- at most one
transfer per ticket
  from_user   UUID NOT NULL,
  to_user     UUID NOT NULL CHECK (to_user <> from_user),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- checkout_db -----------------------------------------------------------
CREATE TABLE orders (
  order_id       UUID PRIMARY KEY,
  hold_id        UUID NOT NULL UNIQUE,           -- one order per hold
  user_id        UUID NOT NULL,
  amount_paise   BIGINT NOT NULL CHECK (amount_paise > 0),
  refunded_paise BIGINT NOT NULL DEFAULT 0 CHECK (refunded_paise BETWEEN 0
AND amount_paise),
```

```
  status         TEXT NOT NULL CHECK (status IN
('PENDING_PAYMENT','PAID_PENDING_CONFIRMATION','CONFIRMED',

'REFUND_PENDING','PARTIALLY_REFUNDED','REFUNDED','FAILED')),
  version        INT NOT NULL DEFAULT 0,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE payments (
  payment_id   UUID PRIMARY KEY,
  order_id     UUID NOT NULL UNIQUE REFERENCES orders,   -- one capture per
order
  provider_ref TEXT UNIQUE,
  status       TEXT NOT NULL CHECK (status IN
('CREATED','SUCCEEDED','FAILED')),
  amount_paise BIGINT NOT NULL CHECK (amount_paise > 0)
);
CREATE TABLE refunds (
  refund_id       UUID PRIMARY KEY,
  order_id        UUID NOT NULL REFERENCES orders,
  amount_paise    BIGINT NOT NULL CHECK (amount_paise >= 0),
  reason          TEXT NOT NULL CHECK (reason IN
('CANCELLED','CONFIRM_REJECTED','EVENT_CANCELLED')),
  status          TEXT NOT NULL CHECK (status IN
('REQUESTED','TICKETS_VOIDED','SENT','COMPLETED','FAILED')),
  provider_ref    TEXT UNIQUE,
  idempotency_key TEXT NOT NULL UNIQUE
);
CREATE TABLE refund_items (                     -- a ticket is refunded at
most once
  refund_id    UUID NOT NULL REFERENCES refunds,
  ticket_id    UUID NOT NULL PRIMARY KEY,
  amount_paise BIGINT NOT NULL CHECK (amount_paise >= 0)
);
CREATE UNIQUE INDEX one_full_refund_per_rejected_order ON refunds (order_id)
WHERE reason = 'CONFIRM_REJECTED';

CREATE TABLE sagas (
  saga_id  UUID PRIMARY KEY,
  kind     TEXT NOT NULL CHECK (kind IN ('CHECKOUT','REFUND')),
  order_id UUID NOT NULL REFERENCES orders,
  state    TEXT NOT NULL,
  version  INT  NOT NULL DEFAULT 0,             -- optimistic concurrency
between workers
```

```
  UNIQUE (kind, order_id, saga_id)
);
CREATE TABLE saga_steps (
  saga_id         UUID NOT NULL REFERENCES sagas,
  step            TEXT NOT NULL,               -- e.g. 'CONFIRM_HOLD',
'SEND_REFUND'
  status          TEXT NOT NULL CHECK (status IN
('PENDING','RUNNING','SUCCEEDED','UNKNOWN','FAILED')),
  attempts        INT  NOT NULL DEFAULT 0,
  next_attempt_at TIMESTAMPTZ,
  last_error      TEXT,
  PRIMARY KEY (saga_id, step)                  -- unique step identity
);

-- in every service that writes state -------------------------------
CREATE TABLE idempotency_keys (
  scope         TEXT  NOT NULL,                -- operation, e.g. 'POST
/holds'
  user_id       UUID  NOT NULL,
  key           TEXT  NOT NULL,
  request_hash  BYTEA NOT NULL,                -- same key + different body →
422
  status        TEXT  NOT NULL CHECK (status IN ('IN_PROGRESS','COMPLETED')),
  response_code INT,
  response_body JSONB,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at    TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (scope, user_id, key)
);
CREATE TABLE outbox (
  id              UUID PRIMARY KEY,
  topic           TEXT  NOT NULL,
  key             TEXT  NOT NULL,
  payload         BYTEA NOT NULL,
  status          TEXT  NOT NULL DEFAULT 'PENDING' CHECK (status IN
('PENDING','IN_FLIGHT','SENT','DEAD')),
  attempts        INT   NOT NULL DEFAULT 0,
  next_attempt_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  locked_by       TEXT,
  locked_until    TIMESTAMPTZ,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  sent_at         TIMESTAMPTZ
);
```

```
CREATE INDEX outbox_ready ON outbox (next_attempt_at) WHERE status IN
('PENDING','IN_FLIGHT');
CREATE TABLE processed_events (event_id UUID PRIMARY KEY, processed_at
TIMESTAMPTZ NOT NULL DEFAULT now());
```

Invariants these tables enforce. `holds.expires_at` is the one canonical expiry; `seats.hold_expires_at` is a mirror written only in the same transaction (create, extend at checkout, release, expire). `processed_events` stops a consumer applying an event twice, and business-level unique keys (`bookings.order_id`, `payments.order_id`, `refund_items.ticket_id`, `hold_ga_allocations` primary key) stop duplicate effects even if dedup is bypassed.

Order state machine (every change is `UPDATE orders SET status = $to, version = version + 1 WHERE order_id = $1 AND status = $from`; any other transition is rejected)

From	To	Trigger
PENDING_PAYMENT	PAID_PENDING_CONFIRMATION	Signed payment-success webhook
PENDING_PAYMENT	FAILED	Payment failed or payment window expired
PAID_PENDING_CONFIRMATION	CONFIRMED	`ConfirmHold` returns CONFIRMED (now or on a later retry)
PAID_PENDING_CONFIRMATION	REFUND_PENDING	`ConfirmHold` returns a definitive REJECTED
REFUND_PENDING	REFUNDED	Refund webhook succeeded
CONFIRMED	PARTIALLY_REFUNDED or REFUNDED	Cancellation refund completed

From	To	Trigger
PARTIALLY_REFUNDED	REFUNDED	Remaining tickets cancelled and refunded

Identity owns `users`, `credentials`, `oauth_accounts` and `signing_keys`; Catalog owns `events`, `venues` and `seat_layouts` (the static map), copied into Booking's `seats` when a sale is published.

Redis keys

Key	Type	Purpose
`wr:{event}:presale`	Sorted set (random score)	Pre-sale arrivals; shuffled into ticket numbers at sale start
`wr:{event}:seq`	Counter	Next queue ticket number after sale start
`wr:{event}:admitted`	Counter (public)	Highest admitted ticket number; only increases; cached at the edge for 1 s
`wr:{event}:user:{user}`	String, TTL	One active queue entry per user
`wr:{event}:epoch`	Counter	Leadership epoch; incremented every time the admission lease is acquired (fencing token)
`wr:{event}:lease`	String `owner:epoch`, TTL 5 s	Admission-controller lease; renewed only by the owner of the current epoch
`wr:{event}:second_chance`	Sorted set (ticket number)	Users waiting after sell-out, offered returned inventory in order
`pow:{nonce}`	String, TTL 2 min	Spent proof-of-work nonces (no replay)
`seatbits:{event}:{section}`	Bitmap	Fast pre-filter: rejects clearly taken seats before Postgres (Postgres stays the truth)

Key	Type	Purpose
`seatmap:{event}:{section}`	String, TTL 2 s + jitter	Cached availability snapshot for page loads
`rl:{scope}:{key}`	Hash, TTL	Token-bucket rate-limit state

The `{event}` part is a Redis Cluster hash tag: every `wr:` key for one event lands in the same slot, so the admission Lua script can check the lease and update the counter atomically. The cost is the single-shard hot spot noted under Capacity estimates.

Kafka topics (each topic is keyed by the entity whose event order matters; consumers stay idempotent regardless)

Topic	Key	Events	Ordered per	Consumers
`booking.holds`	`hold_id`	HoldCreated, HoldExtended, HoldReleased, HoldExpired, HoldConfirmed	Hold	Notifier, analytics
`booking.inventory`	`event_id:section` (SoldOut keyed by `event_id`)	InventoryChanged (seat or bucket deltas), InventoryReturned, SoldOut	Section	Catalog (snapshots), Notifier (SSE deltas), Waiting Room (second chance, sold out)
`booking.tickets`	`ticket_id`	TicketsIssued, TicketCancelled, TicketTransferred, TicketCheckedIn	Ticket	Notifier, analytics
`checkout.orders`	`order_id`	OrderCreated, PaymentSucceeded, PaymentFailed, OrderConfirmed, RefundRequested, RefundIssued, RefundFailed	Order	Notifier, analytics (sagas call Booking directly)

A hold that spans two sections publishes its lifecycle once on `booking.holds` (one partition, in order) and one `InventoryChanged` per section on `booking.inventory`. Every event carries a unique `event_id` and the aggregate's version, so consumers can drop

duplicates and ignore stale versions.

API design

Browsers talk REST (plus SSE) to the gateway; services talk gRPC to each other with propagated deadlines. Every write takes an `Idempotency-Key`; errors use RFC 9457 problem details.

Public REST (via Gateway)

Method and path	Auth	Purpose	Key responses
`POST /v1/auth/signup`, `/login`, `/refresh`, `/verify-email`	none / refresh token	Email/password auth and verification	200 tokens, 401
`GET /v1/auth/google/start`, `/callback`	none	Google OAuth with PKCE	302
`GET /v1/events/{eventID}`	JWT	Event details, sale status, refund policy	200, 404
`GET /v1/events/{eventID}/queue/challenge`	JWT	Proof-of-work challenge (nonce, difficulty)	200
`POST /v1/events/{eventID}/queue`	JWT	Join with a solved challenge; returns a signed queue ticket	201 `{ticket, number}`, 409 already queued
`GET /v1/events/{eventID}/queue/state`	none	Public, cacheable: `{admitted_upto, rate, sold_out}`; the client computes its own position	200, `Cache-Control: max-age=1`
`POST /v1/events/{eventID}/queue/admit`	JWT + queue ticket	Exchange an admitted ticket for an access token	200 token, 425 not yet
`POST /v1/events/{eventID}/second-chance`	JWT + queue ticket	Stay in line after sell-out	201
`GET /v1/events/{eventID}/seats`	access token	Seat map (cached snapshot)	200
`GET /v1/events/{eventID}/seats/stream`	access token	Live seat-map deltas (SSE)	event stream

Method and path	Auth	Purpose	Key responses
`POST /v1/events/{eventID}/holds`	access token	Hold `{seat_ids}`, or `{best_available: {section or tier, quantity}}`, or `{ga: {section, quantity}}`	201 `{hold_id, seats, expires_at, amount}`, 409, 422
`DELETE /v1/holds/{holdID}`	access token	Release a hold	204
`POST /v1/holds/{holdID}/checkout`	access token	Create order and payment intent	201 `{order_id, payment_url}`
`GET /v1/orders/{orderID}`	JWT	Order, tickets, refunds	200
`GET /v1/orders/{orderID}/cancellation-quote`	JWT	Refund amount under the event's policy	200 `{refund_paise, deadline}`
`POST /v1/orders/{orderID}/cancellations`	JWT	Cancel all or some tickets `{ticket_ids}`	202 `{refund_id}`, 409 past cutoff
`GET /v1/tickets/{ticketID}/qr`	JWT (owner)	Current rotating QR payload	200
`POST /v1/tickets/{ticketID}/transfers`	JWT (owner)	Transfer to `{email}` of a verified account	201, 409 limit reached
`POST /v1/scan`	staff JWT	Validate a QR payload and check in	200 `{valid, seat}`, 409 already used
`POST /v1/webhooks/fakepay`	HMAC signature	Payment and refund results	204
`POST /v1/admin/events`, `PUT .../sale`, `POST .../sale:pause`, `.../sale:resume`	admin JWT	Create event, configure, control a live sale	200, 201

Internal gRPC (one proto package per service)

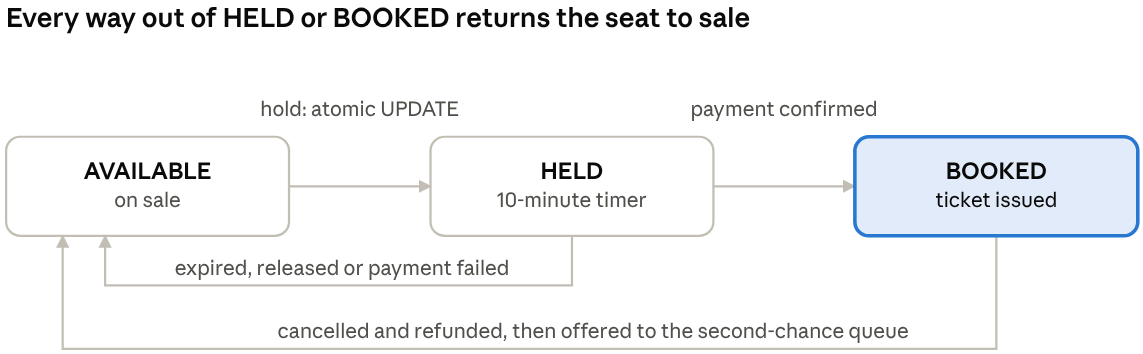
Service	RPCs
`identity.v1`	`GetJWKS`, `VerifyToken`, `LookupUserByEmail` (transfers)
`catalog.v1`	`GetEvent`, `GetSeatMap`, `GetRefundPolicy`, `PublishSale`

Service	RPCs
`waitingroom.v1`	`Join`, `GetState`, `Admit`, `VerifyAccessToken`, `OfferSecondChance`
`booking.v1`	`HoldSeats`, `HoldBestAvailable`, `HoldGA`, `ReleaseHold`, `GetHold`, `StartPayment`, `ConfirmHold (idempotent by order)`, `GetBookingByOrder`, `CancelTickets`, `TransferTicket`, `CheckIn`; server-stream `WatchSeats`
`checkout.v1`	`StartCheckout`, `GetOrder`, `QuoteCancellation`, `Cancel`

Deadlines: gateway gives each request a 1 s budget; internal calls inherit the remainder. Only `UNAVAILABLE` is retried, with jittered backoff.

Key flows

Seven flows carry the product; the checkout and refund sagas are orchestrated by Checkout over gRPC with durable step state, and every step is safe to retry.



Seat lifecycle · 3 states

A seat moves forward only through conditional updates; every way out of HELD or BOOKED returns it to sale and to the second-chance queue.

1. Join the queue and get admitted

1. The client fetches a proof-of-work challenge and spends roughly 1 second of CPU solving it. Cheap for one human, expensive for a bot farm joining thousands of times.

2. `POST /queue`: Waiting Room verifies the solution, burns the nonce, claims `wr:{event}:user:{user}` with `SET NX`, and returns a signed queue ticket `{user, event, number, exp}`. Before sale start the user lands in a shuffle set; at start everyone in it gets a random ticket number; after start numbers come from `INCR`.
3. The client polls `GET /queue/state`, a public response with `{admitted_upto, rate, sold_out}` that is identical for everyone and cached at the edge for 1 s. Position = my number − `admitted_upto`; ETA = position ÷ rate. No per-user server work at all.
4. Admission controller with fencing: an instance becomes leader by running one Lua script that increments `wr:{event}:epoch` and sets `wr:{event}:lease = owner:epoch` only if the lease is free. Every admission tick runs a second Lua script that (a) checks the lease still equals its own `owner:epoch`, (b) raises `wr:{event}:admitted` to `max(current, new)`, and (c) renews the lease, all atomically. A controller that paused past its lease finds a newer epoch and stops; it can never move the counter again.
5. The admission rate rises while booking p99 and DB pool wait stay healthy and halves when they don't (additive increase, multiplicative decrease).
6. When `admitted_upto` ≥ my number, the client calls `POST /queue/admit` once and receives a 15-minute access token bound to user and event.
7. On sell-out, `sold_out` flips in the same cached response; users can opt into the second-chance queue in one tap.

2. Hold seats

1. Gateway verifies the access token and the per-user rate limit.
2. Booking claims the idempotency record `(scope, user_id, key)`: completed → return the stored response; in progress → 409 retry later; same key with a different body → 422.
3. Pre-filter: `seatbits` in Redis rejects seats already shown as taken (one in-memory check instead of a doomed transaction).
4. One transaction per mode, each ending with `INSERT holds`, the hold's detail rows, and outbox events (HoldCreated on `booking.holds`, InventoryChanged per section):
   a. Chosen seats: the conditional `UPDATE seats` (all or nothing) plus `hold_seats` rows. A seat reclaimed from an expired hold also marks that old hold `EXPIRED` in the same transaction.
   b. Best-available: `SELECT ... WHERE status = 'AVAILABLE' ORDER BY row_label, seat_number FOR UPDATE SKIP LOCKED LIMIT 40`, pick the first run of N adjacent seats in memory, then the same conditional update. Concurrent buyers skip each

other's locked rows instead of colliding.

   c. General admission, across buckets: lock buckets with stock using `SELECT ... WHERE remaining > 0 ORDER BY (bucket - r + 32) % 32 FOR UPDATE SKIP LOCKED` (r random), take greedily from each until N is covered, and write one `hold_ga_allocations` row per bucket used (for example 2 from bucket 1 and 2 from bucket 2). If the unlocked buckets hold fewer than N, retry once taking plain `FOR UPDATE` locks in ascending bucket order, which waits instead of skipping and cannot deadlock; if total stock is still short, return 409.

5. Returns 201 with the seats or allocation and `expires_at`, or 409 listing unavailable seats with suggested alternatives.

3. Checkout and payment (saga)

1. `POST /holds/{id}/checkout`: Checkout calls `Booking.StartPayment(hold_id, order_id)`, which moves the hold to `PAYMENT_PENDING` and extends its canonical expiry (and the seat mirrors) to cover the payment window, at most 10 more minutes. Checkout then inserts the order `PENDING_PAYMENT` and creates a FakePay payment intent keyed by `order_id`.
2. Buyer pays on FakePay; FakePay sends a signed webhook to the gateway.
3. Checkout verifies the signature and, in one transaction, records the payment `SUCCEEDED` (deduped on `provider_ref`), moves the order to `PAID_PENDING_CONFIRMATION` and creates a pending `CONFIRM_HOLD` saga step. Payment success is durable before any confirmation attempt.
4. Checkout calls `Booking.ConfirmHold(hold_id, order_id)`. It is idempotent: if a booking already exists for that `order_id`, Booking returns it unchanged. The call has three possible outcomes:

Booking response	Checkout action
CONFIRMED (new or already existing)	Order → `CONFIRMED`; outbox OrderConfirmed
REJECTED: the hold is no longer this order's (expired and released)	Order → `REFUND_PENDING`; start the refund saga with reason CONFIRM_REJECTED
Timeout, `UNAVAILABLE`, or any unknown error	Step → `UNKNOWN`; order stays `PAID_PENDING_CONFIRMATION`; retry with backoff. Never refund on a timeout

5. A reconciler scans orders stuck in `PAID_PENDING_CONFIRMATION` for more than 1 minute and calls `Booking.GetBookingByOrder(order_id)`. A booking found means confirm the order; a definitive rejection means refund; still unreachable means keep retrying and alert after 15 minutes.
6. Notifier consumes OrderConfirmed and TicketsIssued and emails the QR ticket.

4. Expire holds

1. Every 2 s, the expiry worker claims holds in `ACTIVE` or `PAYMENT_PENDING` whose `expires_at` has passed, in batches of 1,000 with `FOR UPDATE SKIP LOCKED`, so several replicas never fight.
2. In one transaction per batch: seats return to `AVAILABLE` (mirror cleared), GA quantities are added back from `hold_ga_allocations` rows whose `released_at` is NULL (then stamped, so nothing is restored twice), holds become `EXPIRED`, and HoldExpired and InventoryReturned events are written to the outbox.
3. The hold path also reclaims expired seats inline, so a stalled worker never blocks inventory.
4. A payment that succeeds after its hold expired is caught by step 4 of the checkout saga: `ConfirmHold` returns REJECTED, and the buyer is refunded.

5. Cancel and refund (saga)

1. Buyer requests a quote: Checkout applies the event's policy (for example 100% until 7 days out, 50% until 24 hours) and returns the amount and deadline.
2. `POST /cancellations` with an Idempotency-Key creates a refund row `REQUESTED` and the saga's first step.
3. Checkout calls `Booking.CancelTickets`: tickets become `CANCELLED`, seats return to `AVAILABLE` (or GA buckets are incremented), QR secrets are revoked, and an InventoryReturned event is written to the outbox, all in one transaction. Refund status moves to `TICKETS_VOIDED`.
4. Checkout sends the refund to FakePay keyed by `refund_id`; the signed webhook marks it `COMPLETED` and adds to `orders.refunded_paise` (checked never to exceed the amount paid).
5. Notifier emails the refund receipt. If FakePay keeps failing, the saga retries with backoff and alerts after 1 hour; tickets stay void, so no one can use them meanwhile.

6. Second-chance offers

1. Waiting Room consumes InventoryReturned events (from expiry, failed payment or cancellation).
2. It pops the next user from `wr:{event}:second_chance` and issues a 5-minute access token scoped to that section; Notifier pushes the offer (SSE or email).
3. The returned seats are reserved for that user with a hold created by the offer, so a faster bot cannot snatch them; unclaimed offers expire and move to the next person.

7. Transfer and gate scan

1. Transfer: Booking checks ownership, that no transfer exists yet and the 24-hour cutoff, then in one transaction changes `owner_user_id`, increments the count, rotates the QR secret and inserts the `ticket_transfers` row. The old owner's QR codes stop working immediately.
2. QR codes: the app shows a code derived from the ticket secret and the current 30-second window (a TOTP-style HMAC). A screenshot goes stale within about a minute, but a code shared live can still be scanned by someone else inside that window. This reduces screenshot resale; it does not prevent sharing.
3. Scan (online only; gates need venue connectivity, offline scanning is out of scope): `POST /scan` recomputes the code for the current and previous window, then runs `UPDATE tickets SET status = 'CHECKED_IN', checked_in_at = now() WHERE ticket_id = $1 AND status = 'VALID'`. One row means admit; zero means already used or cancelled, so a replayed or duplicated code gets in at most once (first scan wins).

Correctness guarantees and failure handling

Four mechanisms carry every guarantee: atomic conditional updates, unique constraints, idempotency keys, and the outbox. Redis and Kafka can fail without ever corrupting seat or money state.

How a double-booking is made impossible

1. A hold is one `UPDATE seats ... WHERE event_id = $1 AND seat_id = ANY($2) AND (status = 'AVAILABLE' OR (status = 'HELD' AND hold_expires_at < now()))` inside a transaction. If rows affected ≠ seats requested, the transaction rolls back: all or nothing.
2. Postgres row locks serialize competing updates; the loser re-checks the `WHERE` clause against the new row and matches nothing. Best-available uses `SKIP LOCKED` only to choose candidates; the conditional update still decides.

3. General admission decrements each bucket only `WHERE remaining >= quantity`, `CHECK (remaining >= 0)` makes a negative count impossible, and every unit taken is recorded per bucket so release restores exactly what was taken, once (`released_at`).
4. Confirmation is idempotent by `order_id` and only succeeds while the hold belongs to that order; an expired and re-sold hold returns a definitive REJECTED.
5. A partial unique index allows at most one live ticket per seat; cancelling a ticket frees the seat for resale without breaking the rule.
6. Money: payment success is persisted before confirmation is attempted, a refund is started only on a definitive rejection or a cancellation (never on a timeout), each ticket can be refunded once (`refund_items`), and refunds can't exceed payments.
7. Leadership: every admission-counter write checks the current lease epoch inside the same Lua script, so a stale controller cannot write.

These are guarantees of the data model and the code paths above; the test suite (T1, T7, T12, T13) checks them repeatedly, but tests are evidence, not a proof of every possible execution.

Failure scenarios

Failure	Effect	Handling
Client retries a hold, checkout, cancel or transfer	Possible duplicate	Idempotency record returns the stored response; different body with the same key → 422
`ConfirmHold` committed but the response was lost (timeout)	Checkout doesn't know the outcome	Step marked UNKNOWN; retry is idempotent and returns the existing booking; reconciler checks by `order_id`. No refund on a timeout
Payment succeeds after the hold expired and seats moved on	Paid, no seats	`ConfirmHold` returns REJECTED → refund saga with reason CONFIRM_REJECTED
Payment or refund webhook lost or duplicated	Order or refund stuck, or processed twice	Reconciler polls FakePay for anything pending > 2 min; `provider_ref` unique dedupes

Failure	Effect	Handling
Refund provider unavailable	Buyer waits for money	Tickets already void; saga retries with backoff; completes within 15 min of recovery; alert after 1 hour
Checkout worker crashes mid-saga	Saga half done	Steps are durable; another replica resumes from the last completed step
Admission controller pauses past its lease	Two controllers believe they lead	Epoch fencing: the stale controller's next write fails the Lua check and it steps down
Booking service crashes mid-request	Unknown outcome for the client	Transaction either committed or not; retry with the same key is safe
Redis down	Queue counter and caches unavailable	Admissions pause (fail closed); admitted users continue with signed tokens; seat map falls back to throttled DB reads; holds skip the bitmap pre-filter
Redis shard for a hot event saturated	Join and status latency rise	Measured ceiling documented; joins shed with 429 + `Retry-After`; edge cache keeps status reads off Redis
CDN or edge cache down	Status polls hit origin	Gateway serves `queue/state` from an in-process 1 s cache; clients back off on 429
Kafka down	Events not published	Outbox rows stay PENDING with retry backoff; relay publishes on recovery; second-chance offers delay but are never lost
Postgres primary fails	Holds, checkouts and refunds fail briefly	Synchronous standby promoted; admission rate drops to zero until healthy
Expiry worker stops	Expired holds linger	Holds reclaim expired seats inline, so inventory is never stuck

Failure	Effect	Handling
FakePay slow or failing	Checkout latency rises	Timeout + circuit breaker; hold extended for the payment window; failures release seats
Bot floods queue joins	Queue polluted	Proof-of-work, one entry per verified account, per-IP limits
QR code shared live or replayed	Someone else tries to enter	Code expires within about a minute; one-time check-in lets only the first scan in
Gate loses connectivity	Scans fail	Out of scope for v1: gates require connectivity; documented as a known limitation
Nightly audit finds a mismatch	Possible bug	Page the owner; the report lists the exact orders, seats and payments involved

Observability and load testing

Every claim on the resume comes from one comparison: the same k6 scenarios run against the naive baseline and against SurgeTix, with the Grafana "Surge Control Room" recording both.

Instrumentation

- Metrics (Prometheus): RED per service; `queue_tickets_issued`, `admitted_upto`, `admission_rate`, `seats_remaining`, `holds_total{mode,result}`, `refunds_total{reason,status}`, `second_chance_offers_total{result}`, `double_booked_seats` and `oversold_ga` (audit queries, must stay 0), DB pool wait, Redis and edge cache hit ratios, Kafka lag, outbox backlog, saga steps stuck > 5 min.
- Efficiency metrics: Postgres round trips per seat sold, origin requests per 1,000 queue polls, CPU-seconds and allocations per booking (pprof), estimated cloud cost per 1,000 tickets sold.
- Traces (OpenTelemetry → Tempo or Jaeger): gateway → service → Postgres/Redis → Kafka → notifier, one trace per request, including each saga step.
- Logs: `slog` JSON with `trace_id`, `user_id`, `event_id`, `order_id`.

Test scenarios

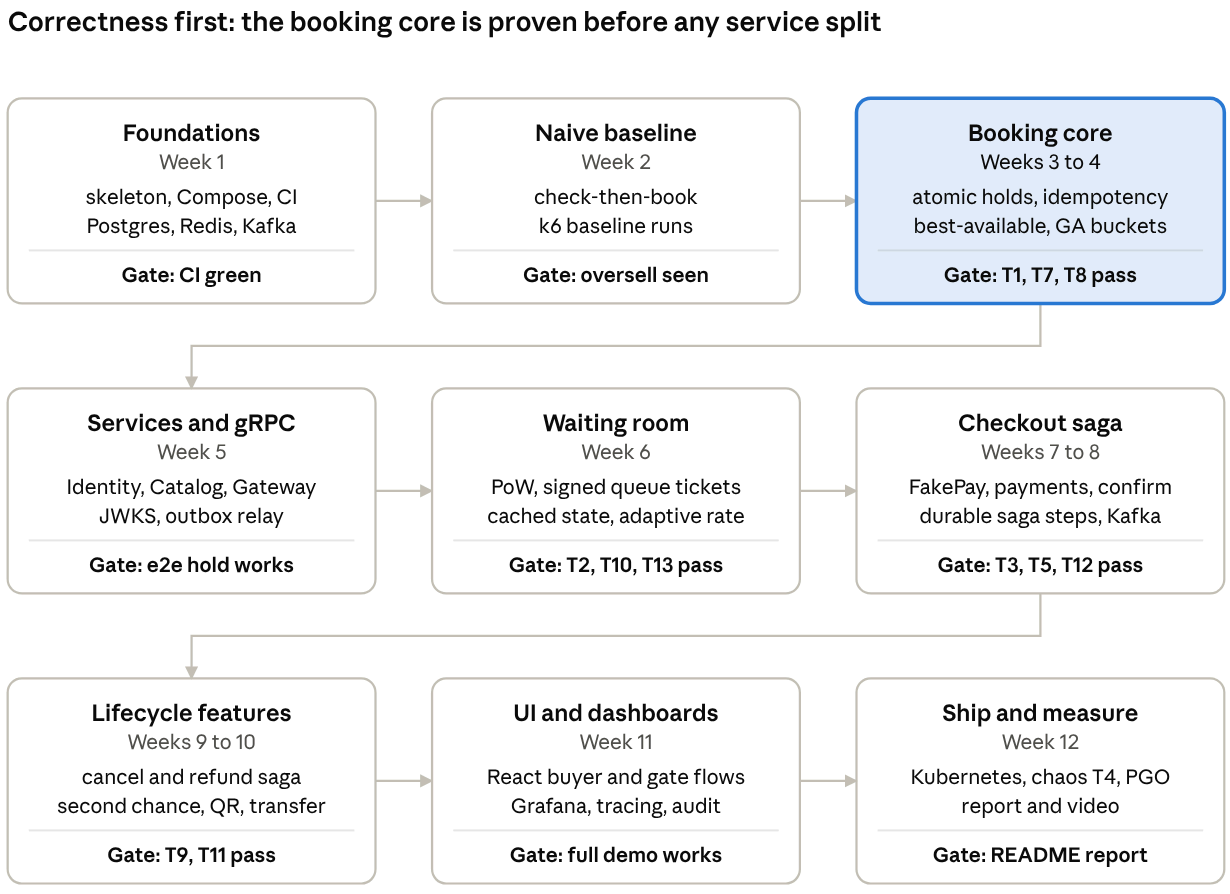
ID	Scenario	Pass criteria
T1	Contention: 5,000 concurrent buyers on the same 500 seats, run 100 times	0 double-booked seats; holds granted ⩽ 500
T2	Spike: 50,000 virtual users join in 60 s and poll the queue; also step one event's join and poll rate up until Redis saturates	Status p99 < 50 ms; ⩾ 95% of polls served from cache; Redis ceiling, CPU, memory and failover time recorded
T3	Full sale: T2 plus admitted users hold, pay and confirm until sold out	Per-step latency targets met; > 99% admitted users complete or see sold-out
T4	Chaos: kill a booking pod, a checkout pod mid-saga and the Kafka broker	No lost bookings or refunds; sagas resume; recovery without manual steps
T5	Slow provider: FakePay adds 3 s latency and 20% failures, then goes down for 5 minutes	Breaker opens; seats released on failure; no double charges; refunds complete within 15 min of recovery
T6	Baseline: T1–T3 against the naive build with the same k6 profile	Recorded for comparison (errors, p99, oversells)
T7	GA contention: 10,000 buyers on a 2,000-ticket GA section, including fragmented stock (no single bucket can fill a 4-ticket order)	0 oversold; multi-bucket orders succeed; released allocations restore exact quantities; lock wait with 32 buckets vs 1 measured
T8	Best-available vs manual picking under the same load	Conflict rate and holds per second compared and reported
T9	Refund storm: 10% of orders cancel within 2 minutes while sales continue	Every refund completes exactly once; returned seats reach second-chance users within 5 s
T10	Bot run: 1,000 scripted joins per second from few IPs	Join rate capped by proof-of-work and limits; human join latency unaffected
T11	Gate: replay a 2-minute-old QR; scan the same ticket twice concurrently	Old code rejected; exactly one check-in succeeds
T12	Lost confirmation: Booking commits `ConfirmHold`, then the response is dropped (fault-injection proxy)	Order ends `CONFIRMED`; 0 refunds issued; reconciler or retry resolves within 2 min

ID	Scenario	Pass criteria
T13	Stale leader: pause the admission leader (SIGSTOP) longer than its lease, then resume	New leader takes over; 0 counter writes from the old epoch

Deliverable: a short report in the README with side-by-side charts (error rate, p99, oversold seats, DB queries per second) and the exact setup (machines, replica counts, versions).

Build phases

The plan builds the broken version first, proves the booking core correct, and only then splits services and adds the waiting room; each phase ends at a test gate from the scenarios above.



Build plan · 9 phases, about 12 weeks part-time

Week counts assume part-time work alongside classes; a phase moves on only when its gate passes, so the timeline stretches rather than skipping correctness work.

Open questions

- [ ] Refund policy defaults: 100% until 7 days out, 50% until 24 hours, none after? Or let each event choose freely?
- [ ] Proof-of-work difficulty: about 1 s on a mid-range phone; tune after measuring on real devices.
- [ ] Postgres access: `pgx` with hand-written SQL, or `sqlc` code generation? Proposed: `sqlc` for CRUD, hand-written SQL for the hold, GA and refund paths.
- [ ] Should a ticket bought through the second-chance queue be transferable, or locked to the buyer?
- [ ] Hosting: demo video and local run only, or a small cloud VM during application season?
- [ ] Load generation: which cloud VMs for the 50,000-user k6 run, and what budget?
- [ ] Should the naive baseline be a single monolith (simplest) or the same services without the waiting room and atomic holds (fairer comparison)?